

SPEECH EMOTION RECOGNITION

Project Report

MFCC + CNN on the RAVDESS Dataset

Author
Parkhi Yadav

Institution
IIT Madras

Programme
B.Sc. Data Science (Undergraduate)

Project Type
Personal Research Project

Abstract

This report documents the design, development, and iterative improvement of a Speech Emotion Recognition (SER) system using MFCC features and a Convolutional Neural Network, achieving 60% accuracy on the 8-class RAVDESS benchmark.

1. Introduction

Speech Emotion Recognition (SER) is the task of automatically identifying human emotional states from audio signals. It sits at the intersection of signal processing, machine learning, and affective computing, with applications ranging from mental health monitoring to voice-controlled interfaces.

This project is a personal, self-directed research effort undertaken as part of my undergraduate studies in Data Science at IIT Madras. The goal was to build, from scratch, a working SER pipeline that is grounded in sound ML engineering — not just a tutorial reproduction.

2. Dataset — RAVDESS

The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) is the standard benchmark for actor-based SER research.

Property	Detail
Speakers	24 professional actors (12 male, 12 female)
Emotions	Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised
Total files	~1,440 .wav audio files
Sample rate	22,050 Hz
Split	80% training / 20% test (random seed 42)

Dataset Note

RAVDESS is actor-based, meaning speakers are professional actors performing emotions rather than spontaneous recordings. This makes it a cleaner benchmark but also means models trained on it may not generalise directly to natural speech. This limitation is acknowledged in the literature and in this project.

3. Methodology & Approach

3.1 Feature Extraction

Raw audio cannot be fed directly into a CNN. Instead, each audio file is transformed into a 2D feature matrix representing acoustic content over time. This project uses a three-channel MFCC representation:

- **MFCC:** MFCC (40 coefficients) — captures the spectral envelope of the signal, approximating how the human auditory system perceives sound.
- **Delta:** Delta MFCC — first-order derivative of the MFCC, capturing how spectral energy changes frame to frame (velocity).
- **Delta-Delta:** Delta-Delta MFCC — second-order derivative, capturing acceleration of spectral change over time.

These three representations are stacked along the frequency axis to produce a (120×174) feature map per audio file. A fixed-length padding strategy (`librosa.util.fix_length`) ensures all inputs share identical dimensions.

3.2 Data Pipeline

- Normalisation: mean and standard deviation computed on training data only, then applied to both train and test sets. This prevents data leakage from test statistics into the training process.
- Augmentation: lightweight Gaussian noise is added exclusively to training samples after the train/test split, ensuring evaluation integrity.
- Class weights: computed automatically from training label frequencies using `sklearn's compute_class_weight`, replacing hardcoded guesses.

3.3 Model Architecture

The model is a lightweight CNN designed to treat the $(120 \times 174 \times 1)$ MFCC feature map as a 2D spatial representation — analogous to an image, where the axes represent frequency and time respectively.

Layer	Output Shape	Parameters
Conv2D(32) + BN + Pool	(120, 174, 32)	448
Conv2D(64) + BN + Pool	(60, 87, 64)	18,752
Conv2D(128) + BN + Pool	(30, 43, 128)	73,856
GlobalAveragePooling2D	(128,)	0
Dense(128) + Dropout(0.4)	(128,)	16,512
Dense(8) + Softmax	(8,)	1,032

Total	—	110,600
-------	---	---------

Key Design Decision: GlobalAveragePooling2D

The most critical architectural choice was replacing Flatten() with GlobalAveragePooling2D(). Flatten after three conv+pool layers produced 75,264 values, leading to 9.6 million parameters in the Dense layer — far too many for a 1,440-sample dataset. GlobalAveragePooling2D reduces this to 128 values, cutting the Dense layer to ~16K parameters and eliminating model collapse.

4. Development Journey

One of the most important aspects of this project is its iterative nature. The path from a broken baseline to a stable result involved multiple failure modes, each of which deepened understanding of the underlying ML engineering.

Phase	Accuracy	Status	Key Change
v1 — Baseline	25–30%	Class bias	Basic MFCC + CNN, no tuning
v2 — First Fix	~10%	Collapse	Aggressive class weights + deltas caused collapse
v3 — Stabilise	~60%	Stable	Simplified model, fixed normalisation
v4 — Bug Fixes	30%	Collapse	Delta features without GlobalAvgPool = 9.6M params
v5 — Final	60%	Solid	GlobalAveragePooling2D + 3 Conv blocks + L2

Phase 1 — Baseline (25–30%)

The initial model used raw 40-coefficient MFCCs with a two-layer CNN and no regularisation. The model was biased toward certain classes (angry, surprised) and ignored others entirely. This phase established the problem scope and confirmed that a basic pipeline could at least learn some patterns.

Phase 2 — First Fixes Lead to Collapse (~10%)

Introducing delta features, aggressive class weights (2.5× on neutral), and more complex architecture simultaneously caused the model to collapse to ~10% accuracy. The model chose a degenerate solution — predicting one class repeatedly. This failure was instructive: it revealed that multiple simultaneous changes make debugging impossible, and that class weight tuning must be data-driven, not intuition-based.

Phase 3 — Stabilisation (~60%)

Reverting to a simplified model, fixing the normalisation pipeline (computing stats on training data only), and removing aggressive class balancing restored stability. The key insight was that the data leakage in normalisation — computing mean/std before the train/test split — was silently inflating validation metrics in earlier runs.

Phase 4 — Second Collapse after Delta Features (30%)

Adding delta and delta-delta features correctly (post-split, training only) tripled the input feature map from (40 × 174) to (120 × 174). This caused the Flatten layer to produce 75,264 values, creating a 9.6M parameter Dense layer on a ~1,440 sample dataset. The model immediately collapsed to predicting 'surprised' for 96% of samples — a classic sign of parameter explosion.

Diagnosing this required reading the `model.summary()` output and calculating the parameter-to-sample ratio.

Phase 5 — Final Architecture (60%)

Replacing Flatten with GlobalAveragePooling2D reduced the Dense layer from 9.6M to ~16K parameters. Adding a third Conv block (128 filters), L2 regularisation, and increased dropout stabilised training. The model now shows genuine per-class learning with angry (F1 = 0.82), surprised (0.79), and disgust (0.71) performing at a competitive level.

5. Results & Analysis

5.1 Classification Report

Emotion	Precision	Recall	F1-Score	Assessment
Angry	0.91	0.74	0.82	Excellent
Surprised	0.85	0.73	0.79	Strong
Disgust	0.66	0.78	0.71	Strong
Fearful	0.62	0.72	0.67	Good
Happy	0.88	0.44	0.59	Moderate
Sad	0.31	0.85	0.46	Weak
Calm	1.00	0.27	0.43	Weak
Neutral	0.40	0.10	0.16	Difficult
Overall	0.73	0.60	0.60	Solid Baseline

5.2 Key Observations

- Strong classes:** High-arousal emotions (angry, surprised, disgust) are learned well. These have distinct prosodic signatures — high energy, rapid rate — that MFCC features capture reliably.
- Hard classes:** Neutral, calm, and sad are acoustically similar — all share low energy, slow tempo, and narrow pitch range. The model systematically confuses these three, which matches findings in published SER literature. This is a feature-space problem, not a model bug.
- Calm anomaly:** Calm achieves precision of 1.00 but recall of only 0.27 — meaning when the model predicts 'calm' it is always correct, but it misses 30/44 calm samples by classifying them as 'sad'. This asymmetry reveals that the decision boundary for calm is very tight.
- Overfitting:** Training accuracy of 98% versus validation accuracy of 60% indicates real overfitting. Early stopping with `restore_best_weights` captures the generalisation peak, but the gap signals that more augmentation variety is needed.

5.3 Comparison to Literature

Approach	Typical Accuracy	Tier
MFCC + SVM (classic baseline)	55–65%	Baseline
MFCC + CNN — this project	60%	Baseline
MFCC + delta + deeper CNN	65–75%	Competitive
Spectrogram + CNN + augmentation	70–80%	Strong

Wav2Vec / HuBERT (transformer)	85–90%+	SOTA
--------------------------------	---------	------

6. Real-World Applications

Speech Emotion Recognition has practical value across multiple high-impact domains:

Domain	Application
Mental Health	Detecting signs of depression, anxiety, or stress from voice patterns in therapy or telehealth settings
Call Centres	Real-time monitoring of customer frustration to escalate calls or trigger agent alerts automatically
Education	Adaptive e-learning platforms that detect student engagement or frustration and adjust content delivery
Automotive	In-vehicle systems that detect driver stress or fatigue from speech to trigger safety interventions
HCI	Emotion-aware virtual assistants that adapt their response tone based on the user's detected emotional state
Social Robots	Companion robots for elderly care that can recognise distress and respond appropriately

7. Future Improvements

The current model is a solid baseline. The following improvements are realistic next steps that do not require transformers or heavy infrastructure:

Improvement	Description	Expected Gain
Pitch shift + time stretch	Augment training set during loading to reduce the train/val gap (98% vs 60%)	+3–5% accuracy
Add ZCR + RMS energy	Append prosodic features alongside MFCC. Helps separate low-arousal classes (calm, neutral, sad)	+3–5% accuracy
t-SNE feature visualisation	Plot MFCC embeddings per emotion to visually show why neutral/calm/sad overlap — research-quality analysis	Research depth
Cross-corpus testing	Test your RAVDESS model on EMODB or CREMA-D to measure generalisation ability	Generalisability
LSTM or BiGRU layer	Replace GlobalAvgPool with a recurrent layer to capture temporal sequence — still lightweight	+5–8% accuracy

Research Opportunity

The clearest research contribution this project could make is a systematic analysis of the neutral/calm/sad confusion. Plotting t-SNE embeddings of the MFCC feature space per emotion class, and showing visually that these three clusters overlap, would constitute a research-level finding — not just a performance number.

8. Technical Stack

Component	Tool / Library
Language	Python 3.12
Audio Processing	librosa — loading, MFCC, delta, fix_length
Deep Learning	TensorFlow 2 / Keras — model, training, callbacks
ML Utilities	scikit-learn — LabelEncoder, train_test_split, classification_report, compute_class_weight
Visualisation	matplotlib, seaborn — confusion matrix, training curves
Version Control	Git + GitHub
IDE	Visual Studio Code

9. Conclusion

This project demonstrates that a well-engineered lightweight CNN with MFCC features can achieve competitive performance on 8-class speech emotion recognition on RAVDESS, matching the MFCC + SVM baseline tier documented in the research literature.

More importantly, the development journey — from a broken baseline through two distinct model collapse events to a stable 60% result — reflects the real practice of machine learning engineering: diagnosing failure modes, understanding their root causes, and applying principled fixes rather than guesswork.

The two most significant technical contributions of this project are: (1) identifying and correcting data leakage in the normalisation pipeline, and (2) diagnosing the 9.6M parameter explosion caused by Flatten on a tripled feature map, and resolving it with GlobalAveragePooling2D. Both fixes demonstrate research-level attention to ML engineering correctness.

Final Performance Summary

Overall Accuracy: 60% | Macro F1: 0.58 | Best Class: Angry (F1 = 0.82) | Model Parameters: 110,600 | Dataset: RAVDESS (1,440 samples, 8 emotions, 24 actors)